

# Augur: Similarity-Enabled Sparse I/O for Efficient Prefix-KV Inference

Jing Zou<sup>1,\*</sup> Shangyu Wu<sup>2,\*</sup> Hancong Duan<sup>1</sup> Qiao Li<sup>2,†</sup> Chun Jason Xue<sup>2</sup>

<sup>1</sup>UESTC <sup>2</sup>MBZUAI

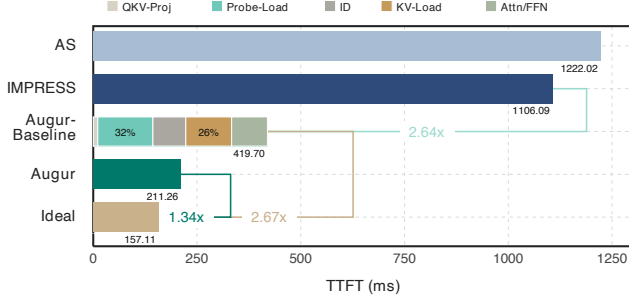


Figure 1: TTFT of existing SSD-offloaded prefix-KV inference system and the compute-only lower bound.

## Introduction and Motivation

LLM applications reuse long prefixes across RAG, conversational assistants, and agentic workloads. Prefix KV caches often exceed HBM capacity and are offloaded to CPU memory or SSD [1, 2], whose lower bandwidth makes KV loading dominate time-to-first-token (TTFT).

To reduce this loading cost, state-of-the-art systems such as IMPRESS [2] apply sparse attention and load only the critical subset of KV entries. Yet at a 10% budget this yields only a 1.1 $\times$  TTFT speedup over full-cache loading [1] (Figure 1). One cause is the mismatch between token-level selection and block-level I/O loading. Aligning selection and storage at chunk granularity (*Augur-Baseline*) removes it and achieves a 2.64 $\times$  speedup, yet a considerable gap to the *Ideal* remains.

This gap has two structural sources. First, prefix-cached prefill is bound by a strict data dependency, as both probing and attention consume KV data that must be loaded first. I/O therefore never proceeds while the GPU computes. Second, probing itself is expensive with modern GQA models, since identifying critical chunks still requires scanning the whole sequence, even with fewer heads (Figure 1).

## Method

We propose Augur, which exploits the similarity of critical sets across layers to break the load-compute dependency and shortens the probing that remains on the critical path.

**Predictive reuse.** We observed that adjacent layers share 72–75% of their critical chunks and only about 1.5% are genuinely new. Therefore, Augur predicts each layer’s critical set from a per-request importance table rather than re-identifying it.

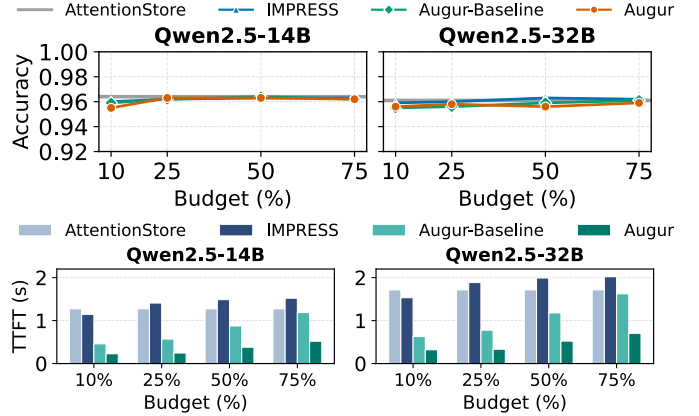


Figure 2: Model performance of different inference systems on the representative dataset (SST-2).

**Hierarchical prefetching.** The predicted set lets Augur load KV before it is needed. Augur groups consecutive layers into periods and prefetches KV for the remaining layers of the current period and for the next period while computation proceeds, fetching only mispredictions on demand.

**Cascaded probing.** Augur screens chunks with compact aggregated keys and loads token-level keys only for a small candidate set. Identifying critical chunks therefore no longer requires reading the whole prefix.

## Overview of Results

**Setup.** We evaluate Qwen2.5-14B/32B on four shared-prefix workloads with 16K-token prefixes and KV budgets from 10% to 75%, comparing Augur against AttentionStore, IMPRESS, and Augur-Baseline on a common implementation.

**Results.** Across four workloads and both model sizes, Augur stays within 1.2 points of full-cache task quality, achieving TTFT speedups of 2.8–6.2 $\times$  over IMPRESS and about 2 $\times$  over Augur-Baseline across budgets. At a 10% budget, speedups over IMPRESS and AttentionStore are 4.9 $\times$  and 5.4 $\times$ , respectively. Figure 2 shows SST-2 results.

## References

- [1] Bin Gao et al. 2024. Cost-Efficient Large Language Model Serving for Multi-turn Conversations with CachedAttention. In *USENIX ATC*. 111–126.
- [2] Weijian Chen et al. 2025. IMPRESS: An Importance-Informed Multi-Tier Prefix KV Storage System for Large Language Model Inference. In *FAST*. 187–201.

\*These authors contributed equally. <sup>†</sup>Corresponding author.